Page 1 — AI Engineering

Artificial intelligence is becoming an important part of modern software engineering. Building useful AI systems requires more than simply calling a language model. An engineer needs to understand how data is collected, processed, represented, retrieved, and finally provided to the model.

A practical example is a document question-answering system. A company may have hundreds of internal documents containing policies, procedures, technical information, and reports. Instead of searching through every document manually, the system can process the documents, split them into useful sections, create vector representations, and store them in a vector database.

When a user asks a question, the system retrieves the most relevant information and provides it to a language model as context. The model can then generate an answer based on the retrieved material. This general approach is known as retrieval-augmented generation, or RAG.

A strong engineering project should demonstrate the complete pipeline. It can include document ingestion, text processing, embeddings, vector search, retrieval, prompt construction, model interaction, an API, and evaluation. The goal is not to build the largest possible system, but to understand each layer well enough to make sensible engineering decisions.

Page 3 — Machine Learning Engineering

Machine learning engineering is a process of turning mathematical ideas into systems that solve practical problems. A model is only one part of that system. Data quality, training procedures, evaluation, deployment, monitoring, and user experience can all determine whether the final application is useful.

For neural networks, concepts such as activation functions, optimization, regularization, overfitting, validation, and evaluation metrics provide the foundation for understanding how models learn. For language models, additional ideas such as tokenization, embeddings, attention, fine-tuning, quantization, and retrieval become important.

Fine-tuning is useful when a model needs to learn a particular behavior, format, or domain style that is difficult to obtain through prompting alone. Techniques such as LoRA and QLoRA can make adaptation more efficient by reducing the number of parameters that must be updated or by reducing the memory required during training.

Engineering judgment is often more important than simply knowing many tools. A good engineer first identifies the problem, defines what success means, chooses an appropriate architecture, and then measures its performance. Understanding why a component is being used is more valuable than simply memorizing its name.